Оценка ролевой сегрегации в языковых моделях методом интроспекции мыслей

Даниил Привезенцев[1*]

[1*]Национальный исследовательский университет «Высшая школа экономики», Москва, Россия.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Аннотация

Рост числа агентных сценариев поверх больших языковых моделей делает стоимость инференса, объем контекста и надежность промежуточных решений центральными ограничителями практического применения AI-ассистентов в разработке программного обеспечения. В статье предлагается научно строгая переработка исходной постановки задачи оценки ролевой сегрегации на основе метода Introspection of Thought (INoT). В отличие от черновых версий работы, статья не выдает неподтвержденные архитектурные claims и не фабрикует экспериментальные результаты. Вклад работы состоит в четырех элементах: (i) аудите литературы с исключением слабых и непроверяемых источников; (ii) формализации гибридной агентной архитектуры, в которой роли *planner*, *worker*, *critic*, *self-check*, *memory/compression* и *policy controller* координируются вокруг внутреннего интроспективного цикла; (iii) введении системы метрик качества, токенной эффективности, стоимости, надежности и поддерживаемости кода, включая метрики *utility-per-token*, *quality-per-dollar*, *verified-code-rate* и *maintainability-aware success*; (iv) разработке воспроизводимого экспериментального протокола и синтетического, но воспроизводимого раздела по совокупной стоимости владения (TCO). Аналитически показано, что интроспективная архитектура выигрывает у классической внешне оркестрируемой мультиагентной схемы тогда, когда снижение повторной передачи контекста и уменьшение вероятности дорогого перезапуска превосходят стоимость внутренних раундов критики, компрессии и самопроверки.

Keywords: Большие языковые модели, AI agents, Introspection of Thought, Software Engineering, Token Efficiency, Economics of Inference, Code Reliability

1 Введение

Современная разработка программного обеспечения редко сводится к линейной последовательности действий. Разработчик постоянно возвращается к коду, системе контроля версий, описанию задач, логам CI и документации; именно такие переключения, как показывают эмпирические исследования Ko et al. [2], Parnin и Rugaber [3], а также Gonçalves et al. [4], повышают когнитивную нагрузку и замедляют возобновление прерванной работы. На этом фоне ассистенты на основе больших языковых моделей воспринимаются не просто как чат-интерфейс, а как потенциальный способ сократить цикл «намерение–действие–проверка» в инженерной практике.

Однако переход от консультативной модели к агентному контуру меняет характер издержек. В классических схемах ReAct [6] и современных software-engineering агентах вроде SWE-agent [17] большая модель многократно участвует в цикле «мышление–действие–наблюдение», роли обмениваются контекстом, а промежуточные артефакты накапливаются по мере итераций. Для задач, где требуется реальное использование инструментов и изоляция привилегий, такой дизайн оправдан. Но для ревью, локального рефакторинга, документного анализа и объяснения уже найденного решения внешняя мультиагентность может оказаться дороже, чем выигрывает.

Отсюда возникает идея перенести часть ролевого взаимодействия внутрь одного контекста модели. Работы Wei et al. [5], Yao et al. [6], Madaan et al. [7] и Shinn et al. [8] показали, что промежуточные шаги, самокритика и повторная генерация действительно способны повышать качество на этапе инференса. Работа Sun и Zeng [1] делает следующий шаг и описывает *Introspection of Thought* (INoT) как внутренне развернутый ролевой цикл. Но отсюда не следует, что любой промежуточный текст можно трактовать как достоверное объяснение скрытого вычислительного процесса: на эту методологическую границу прямо указывают Turpin et al. [9]. Именно поэтому в настоящей статье интроспекция мыслей понимается не как доступ к «настоящему мышлению» модели, а как управляемый инженерный механизм селекции кандидатов, критики, самопроверки и селективного перезапуска.

Настоящая статья является методологическим слоем более крупного проекта по разработке ассистента разработчика, интегрированного со средой программирования. Полнофункциональная система с IDE-расширением, инструментальными агентами и полной телеметрией на момент написания еще не завершена; поэтому ниже речь идет не об отчете о завершенных натурных экспериментах, а о строгой формализации метода, о планируемом протоколе сравнения и о тех ожидаемых эффектах, которые в дальнейшем будут проверяться на реализованном прототипе.

Центральная исследовательская проблема статьи формулируется так: может ли агентная архитектура с внутренней интроспекцией ролей повысить отношение полезности к токенам и к стоимости инференса без ухудшения качества и инженерной надежности по сравнению с одношаговым одноагентным режимом и классической мультиагентной схемой с внешней оркестрацией?

Вклад работы состоит в четырех частях. Во-первых, выполняется аудит исходной библиографии и удаляются неподтвержденные архитектурные утверждения. Во-вторых, формализуется гибридная архитектура агента с ролями *planner*, *worker*, *critic*, *self-check*, *memory/compression* и *policy controller*. В-третьих, вводятся аналитическая модель стоимости и система метрик, охватывающая качество, токенную эффективность, денежную стоимость, надежность и поддерживаемость кода. В-четвертых, задаются воспроизводимый будущий протокол оценки и прозрачная TCO-рамка, которую можно будет инстанцировать после завершения прототипа.

2 Связанные работы и позиционирование

2.1 От пошаговых рассуждений к интроспективному тестовому улучшению

Работа Wei et al. показала, что явное порождение промежуточных шагов рассуждения может заметно улучшать решение сложных задач [5]. ReAct объединяет рассуждение и действие во внешней среде, позволяя чередовать текстовые шаги и вызовы инструментов [6]. Self-Refine переносит цикл генерации, обратной связи и исправления в рамки одной модели [7], а Reflexion добавляет использование языковой обратной связи и рефлексивной памяти в агентном контуре [8]. На этой линии Sun и Zeng предлагают INoT как более явно структурированный внутренний дебат ролей [1].

Для нашей постановки важны два вывода. Во-первых, тестовое улучшение качества без дообучения действительно возможно за счет промежуточной самокритики. Во-вторых, промежуточный текст не должен трактоваться как гарантированно верное объяснение вычислительного процесса модели. Поэтому в данной статье интроспекция мыслей рассматривается операционально: как управляемый механизм генерации кандидатов, критики и селективного перезапуска, а не как прямое наблюдение скрытого мышления модели.

2.2 Мультиагентные системы для программной инженерии

Классические работы Weiss и Wooldridge формируют базовую теоретическую рамку для мультиагентных систем [10, 11]. В прикладном LLM-контексте роль-специализированные системы получили широкое развитие в работах MetaGPT и ChatDev [12, 13]. Эти подходы показывают, что разделение ролей на архитектора, программиста и тестировщика может улучшать декомпозицию сложных задач, но одновременно создает накладные расходы на межагентную коммуникацию, синхронизацию и повторную передачу состояния.

Систематические обзоры фиксируют быстрый рост числа LLM-ориентированных мультиагентных систем и одновременно подчеркивают их проблемы: ошибка одной роли легко распространяется по цепочке, а стоимость координации возрастает с длиной взаимодействия [14, 15]. Для программной инженерии это особенно заметно в задачах, где несколько ролей читают одни и те же длинные фрагменты кода или логов. Параллельно benchmarks семейства

SWE-bench и системы типа SWE-agent демонстрируют, что реальная автоматизация software engineering требует сочетания длинного контекста, редактирования файлов, вызовов тестов и повторных итераций [16, 17].

2.3 Компрессия контекста, самопроверка кода и поддерживаемость

Для длинных контекстов критично, что увеличение числа токенов повышает не только стоимость, но и риск информационного шума. LongLLMLingua и LLMLingua-2 показывают, что компрессия промпта может одновременно снижать задержку и поддерживать или даже улучшать качество на длинном контексте [20, 21]. Для программной инженерии эта идея получила прямое развитие в работе Jia et al., где компрессия кодового контекста для issue resolution уменьшает токенный бюджет и улучшает результативность на SWE-bench [22]. Самопроверка кода не сводится к языковой рефлексии: compiler feedback, тесты и статический анализ должны оставаться внешними и наблюдаемыми сигналами [23–25].

2.4 Публично документированные архитектурные паттерны современных систем

Для позиционирования статьи важно не просто перечислить популярные модели, а отделить то, что действительно раскрыто первичными источниками, от предположений о скрытой внутренней реализации. Таблица 1 суммирует публично документированные паттерны по состоянию на 13 апреля 2026 года. Она не доказывает отсутствие тех или иных внутренних механизмов у перечисленных систем; она лишь фиксирует границы того, что можно корректно использовать в академическом тексте как подтвержденный факт.

2.5 Исследовательский зазор и научная новизна

Из таблицы 1 следует важный вывод. Современные публично описанные системы уже используют длинный контекст, tool use, reasoning-режимы, browser/terminal interaction, MoE и в ряде случаев interleaved thinking. Следовательно, исследовательский зазор нельзя формулировать как простое отсутствие агентности или рассуждения у индустриальных моделей. Более узкий и содержательный зазор состоит в другом: в открытой литературе пока почти нет работ, где *внутренняя ролевая сегрегация в одном контексте* была бы одновременно (i) формализована как отдельная архитектурная конфигурация, (ii) сопоставлена с внешней мультиагентностью на одинаковых задачах, и (iii) оценена через токены, стоимость, надежность кода и поддерживаемость.

Научная новизна предлагаемого подхода поэтому лежит не в утверждении, что «никто раньше не использовал промежуточные рассуждения», и не в заявлении о заведомом превосходстве INoT над всеми альтернативами. Новизна состоит в том, что INoT интерпретируется как инженерный механизм управления траекторией инференса: он задает, когда выгодно породить несколько кандидатов, когда достаточно внутренней критики, когда нужен внешний проверяющий контур и

Таблица 1 Публично документированные подходы и границы раскрытия у современных агентно-ориентированных систем

Система	Что документировано первичными источниками	Что не следует утверждать как установленный факт
DeepSeek-V3	Технический отчет DeepSeek-V3 документирует MoE-архитектуру, Multi-head Latent Attention и ориентацию на cost-efficient training/inference [26].	Публичные источники не раскрывают внутренний INoT-подобный ролевой дебат или скрытую мультиагентную оркестрацию как основной механизм инференса.
DeepSeek-R1	Технический отчет DeepSeek-R1 документирует reasoning-oriented post-training и RL-схему для усиления способности к рассуждению [27].	Нельзя утверждать, что продуктовые агенты DeepSeek используют конкретную внутреннюю схему распределения ролей без внешнего подтверждения.
Gemini 3.1 Pro	Официальный анонс Google от 19 февраля 2026 года и актуальные Gemini API docs описывают Gemini 3.1 Pro как reasoning-oriented модель для сложных задач с agentic и coding-возможностями в preview [30, 31].	Открытые материалы не раскрывают внутреннюю роль-архитектуру и не описывают INoT-эквивалент как зафиксированный инференсный механизм.
Claude Sonnet 4.5	Публичные материалы Anthropic подтверждают agentic/computer-use capabilities модели, а документация по extended thinking описывает interleaved thinking в связке с tool use [32, 33].	Из этого не следует, что внутри одного вызова реализован именно ролевой дебат planner/worker/critic в смысле INoT.
GLM-4.5	Официальная документация и technical report описывают GLM-4.5 как native agentic foundation model с MoE-архитектурой, 128K контекстом и усилением reasoning/coding/agentic performance [28, 29].	Публично не доказано, что GLM-4.5 использует внутренний мультиагентный спор как основную вычислительную схему.
MiniMax-M2.7	Официальные docs MiniMax выделяют M2.7 for AI Coding Tools и отдельно документируют tool use и interleaved thinking как публично доступные возможности платформы [34].	Документация не дает оснований утверждать наличие INoT-подобного ролевого спора внутри одного вызова модели.
Xiaomi MiMo-V2-Pro	Официальная страница Xiaomi MiMo позиционирует MiMo-V2-Pro как foundation model for the agentic era и раскрывает публичные product-level параметры: 1T total / 42B active parameters и 1M context [35].	Использовать эти данные как доказательство конкретной внутренней схемы планирования, дебата ролей или обучения с интроспекцией нельзя: техническая архитектура раскрыта лишь частично.
OpenAI / ChatGPT agent	OpenAI публично подтверждает unified agentic system, browser use, terminal, connectors и system card для ChatGPT agent [36, 37]. Дополнительно OpenAI публикует SDK-паттерны handoff и общие agent workflow [38, 39].	Внутренняя декомпозиция ролей самого ChatGPT agent вне продуктового описания не раскрыта, поэтому любые детали о скрытом multi-agent decomposition не должны использоваться как факт.

когда следует делать повторный запуск только по подтвержденным ошибкам. В такой постановке вклад статьи является сравнительным и методологическим: она предлагает проверяемую модель гибридной агентной архитектуры и набор метрик, позволяющих в дальнейшем эмпирически ответить, когда внутренний ролевой цикл действительно дешевле и не хуже по качеству, а когда классическая внешняя мультиагентность остается предпочтительной.

3 Научная проблема, гипотезы и формальная постановка

Пусть задана задача $x \in \mathcal{X}$, исходный контекст $\mathcal{C}_0$, множество разрешенных инструментов $\mathcal{Z}$ и ограничения по качеству, времени и стоимости. Агентная система должна построить ответ или патч y, максимизируя качество $Q(y, x)$ и одновременно минимизируя токенный расход T, денежную стоимость C и риск ненадежного изменения кода.

Мы рассматриваем три класса архитектур: **Single-Large**, **Classical-MAS** и **Hybrid-INoT**. Гипотеза **H1** состоит в том, что при длинном общем контексте Hybrid-INoT дает более высокий *utility-per-token*, чем Classical-MAS, за счет уменьшения повторной передачи контекста. Гипотеза **H2** утверждает, что малые вспомогательные модели для компрессии и самопроверки экономически оправданы, если снижают вероятность дорогого повторного вызова крупной модели сильнее, чем стоят сами. Гипотеза **H3** состоит в том, что преимущество Hybrid-INoT уменьшается на задачах, где действительно необходимы параллельные вызовы инструментов, изоляция привилегий или длительная внешняя среда исполнения.

4 Метод

4.1 Архитектура ролей и поток данных

Предлагаемая система использует роли *policy controller*, *memory/compression*, *planner*, *worker*, *critic* и *self-check*. В данной статье эти роли рассматриваются как функциональные позиции внутри управляемого цикла рассуждения, а не как требование запускать каждую роль в отдельном процессе. Некоторые роли могут оставаться внешними сервисами; другие могут быть схлопнуты в разные фазы одного вызова модели.

Сначала *policy controller* очищает вход и проверяет, разрешены ли инструментальные действия. Затем *memory/compression* извлекает релевантный контекст $\hat{\mathcal{C}} \subseteq \mathcal{C}_0$ и убирает шум длинного контекста. После этого *planner* выбирает либо внутренний интроспективный режим, либо внешний агентный режим. Внутри интроспективного режима *worker* порождает K кандидатов, *critic* оценивает их по полезности и риску, а *self-check* либо принимает лучший кандидат, либо инициирует селективный повтор только для тех компонентов, которые не прошли верификацию. В более широком проектном контуре это ядро рассуждения встраивается в IDE-расширение, локальный оркестратор, слой инструментальных адаптеров и журнал аудита, как показано на рис. 1.

4.2 Формальные обозначения

Используем следующие обозначения:

$x \in \mathcal{X}$	задача,
$g(x) \in \mathcal{G}$	класс задачи,

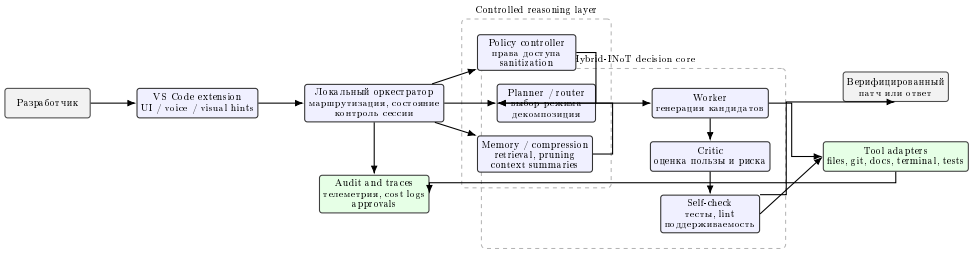

Рис. 1 Архитектурная схема Hybrid-INoT-агента в контуре IDE, локального оркестратора и инструментальных адаптеров

$\mathcal{C}_0$	исходный контекст,
$\hat{\mathcal{C}}$	сжатый контекст,
K	число кандидатов,
I	максимальное число итераций,
p_i^L, r_i^L	входные и выходные токены крупной модели,
p_i^S, r_i^S	входные и выходные токены малой модели,
c_L^{in}, c_L^{out}	цены токена крупной модели,
c_S^{in}, c_S^{out}	цены токена малой модели,
ρ	вероятность дорогого повторного запуска,
Q	качество решения, нормированное в $[0, 1]$.

Общая токенная стоимость траектории определяется как

$$T = \sum_{i=1}^{I}(p_i^L + r_i^L) + \sum_{i=1}^{I}(p_i^S + r_i^S) + T_{\text{tool}}. \tag{1}$$

Для классической внешне оркестрируемой мультиагентной схемы:

$$T_{\text{MAS}} = \sum_{i=1}^{I}\sum_{a\in\mathcal{A}}(p_{i,a} + r_{i,a}) + D_{\text{ctx}} + H_{\text{coord}}, \tag{2}$$

где D_{ctx} — стоимость повторной передачи одних и тех же фрагментов контекста разным агентам, а H_{coord} — накладные расходы внешней координации.

Для предлагаемой Hybrid-INoT архитектуры:

$$T_{\text{INoT}} = T_{\text{core}} + T_{\text{cmp}} + T_{\text{check}} + T_{\text{rerun}}, \tag{3}$$

где T_{core} — основной внутренний раунд ролевого дебата, T_{cmp} — компрессия контекста, T_{check} — селективная проверка, T_{rerun} — стоимость повторных запусков только при провале верификации.

$$\mathbb{E}[T_{\mathrm{MAS}} - T_{\mathrm{INoT}}] > 0 \iff \mathbb{E}[D_{\mathrm{ctx}} + H_{\mathrm{coord}}] > \mathbb{E}[T_{\mathrm{cmp}} + T_{\mathrm{check}} + T_{\mathrm{rerun}} - T_{\mathrm{core}}^{\mathrm{saved}}], \quad (4)$$

4.3 Денежная стоимость и условие выгодности малых моделей

Денежная стоимость инференса задается как

$$C_{\mathrm{inf}} = c_L^{in} \sum_i p_i^L + c_L^{out} \sum_i r_i^L + c_S^{in} \sum_i p_i^S + c_S^{out} \sum_i r_i^S. \quad (5)$$

Если малая модель используется только как фильтр самопроверки, ее применение выгодно тогда, когда она снижает ожидаемую стоимость повторного запуска крупной модели:

$$C_{\mathrm{check}}^S < (\rho_0 - \rho_1)\, C_{\mathrm{rerun}}^L, \quad (6)$$

где ρ_0 и ρ_1 — вероятности повторного запуска без и с малой самопроверкой соответственно, а C_{rerun}^L — ожидаемая стоимость такого повторного запуска.

4.4 Рабочий процесс и алгоритмы

4.4.1 Визуализация рабочего процесса

На рис. 2 представлен основной цикл Hybrid-INoT в виде UML-диаграммы активности. По сравнению с одним только псевдокодом схема явно показывает три точки принятия решения: маршрутизацию между внутренним и внешним исполнением, прохождение или провал самопроверки и повторный запуск только до тех пор, пока сохраняется итерационный бюджет.

4.4.2 Алгоритмическая спецификация

На рис. 3 показана та же логика в виде отдельной блок-схемы. Здесь ключевым является не сам факт повторного запуска, а его селективность: повтор формируется только по подтвержденным сигналам неудачи.

Рис. 4 подчеркивает, что компрессия в предлагаемом методе не сводится к механическому усечению контекста: сначала выделяются релевантные фрагменты, затем проверяется сохранение смысловой целостности.

На рис. 5 видно, что самопроверка опирается на наблюдаемые инженерные сигналы: тесты, статический анализ и прокси поддерживаемости агрегируются в единый верификационный verdict.

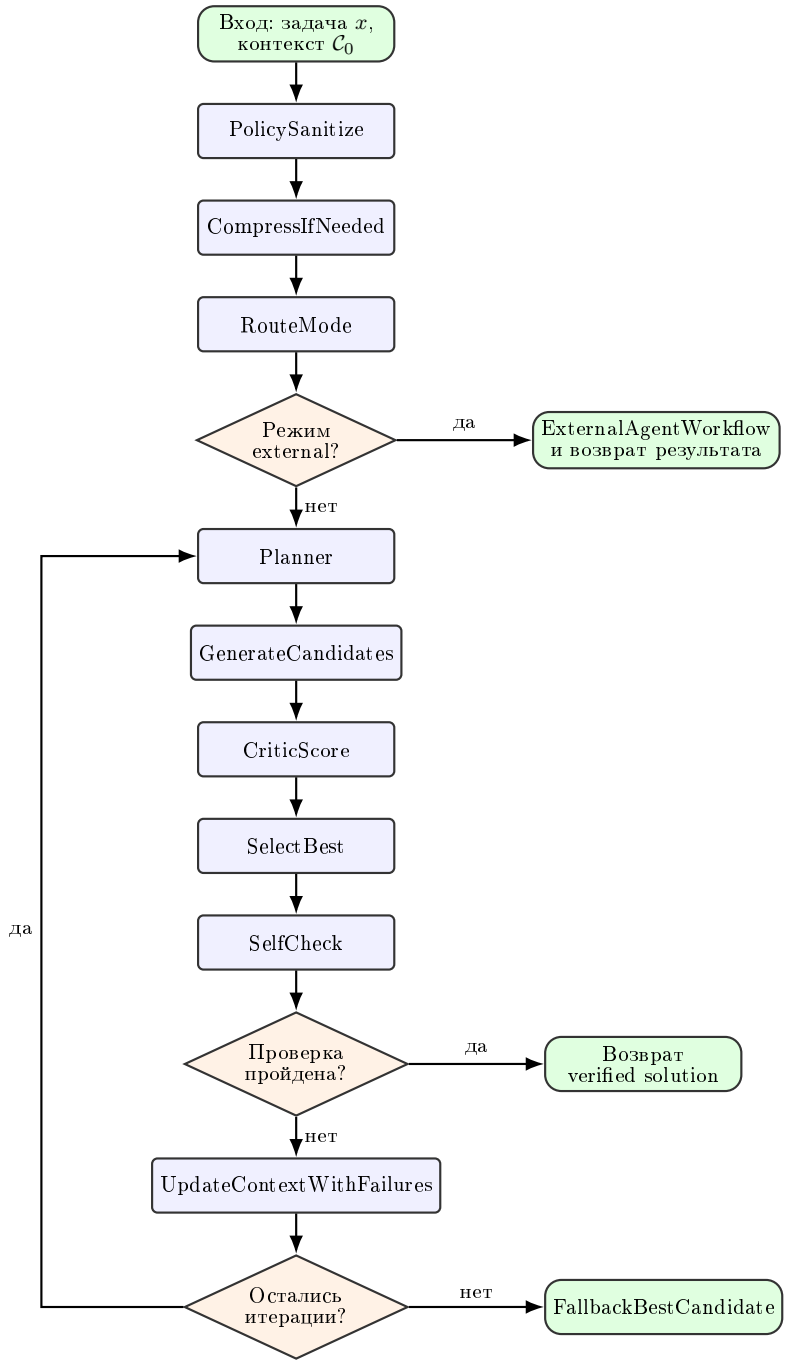

Рис. 2 UML-диаграмма активности интроспективного рабочего процесса Hybrid-INoT

Algorithm 1 Гибридный интроспективный цикл

Require: задача x, контекст $\mathcal{C}_0$, бюджет токенов B, максимум итераций I
Ensure: итоговое решение $y^\star$

$\mathcal{C} \leftarrow \textsc{PolicySanitize}(x, \mathcal{C}_0)$
$\hat{\mathcal{C}} \leftarrow \textsc{CompressIfNeeded}(\mathcal{C})$
$m \leftarrow \textsc{RouteMode}(x, \hat{\mathcal{C}})$
if $m = \texttt{external}$ **then**
    return $\textsc{ExternalAgentWorkflow}(x, \hat{\mathcal{C}})$
end if
for $i = 1$ to I **do**
    $\pi_i \leftarrow \textsc{Planner}(x, \hat{\mathcal{C}}, B)$
    $\mathcal{Y}_i \leftarrow \textsc{GenerateCandidates}(\pi_i, K)$
    $\mathcal{S}_i \leftarrow \textsc{CriticScore}(\mathcal{Y}_i)$
    $y_i \leftarrow \textsc{SelectBest}(\mathcal{Y}_i, \mathcal{S}_i)$
    $v_i \leftarrow \textsc{SelfCheck}(y_i)$
    if $v_i = \texttt{pass}$ **then**
        return y_i
    else
        $\hat{\mathcal{C}} \leftarrow \textsc{UpdateContextWithFailures}(\hat{\mathcal{C}}, v_i)$
    end if
end for
return $\textsc{FallbackBestCandidate}()$

Algorithm 2 Селективный множественный подход с повторным запуском по необходимости

Require: набор кандидатов $\mathcal{Y} = \{y_1, \ldots, y_K\}$, оценки критика s_k, бюджет повторов R
Ensure: отфильтрованный кандидат $y^\star$

Отсортировать $\mathcal{Y}$ по убыванию s_k
Оставить верхние $M \leq K$ кандидатов
for каждый y_m из верхнего множества **do**
    $q_m \leftarrow \textsc{ExternalVerify}(y_m)$
    if $q_m = \texttt{pass}$ **then**
        return y_m
    end if
end for
if $R > 0$ **then**
    Сформировать новый промпт только по выявленным ошибкам
    return $\textsc{SelectiveRerun}(R - 1)$
end if
return кандидат с максимальным s_k

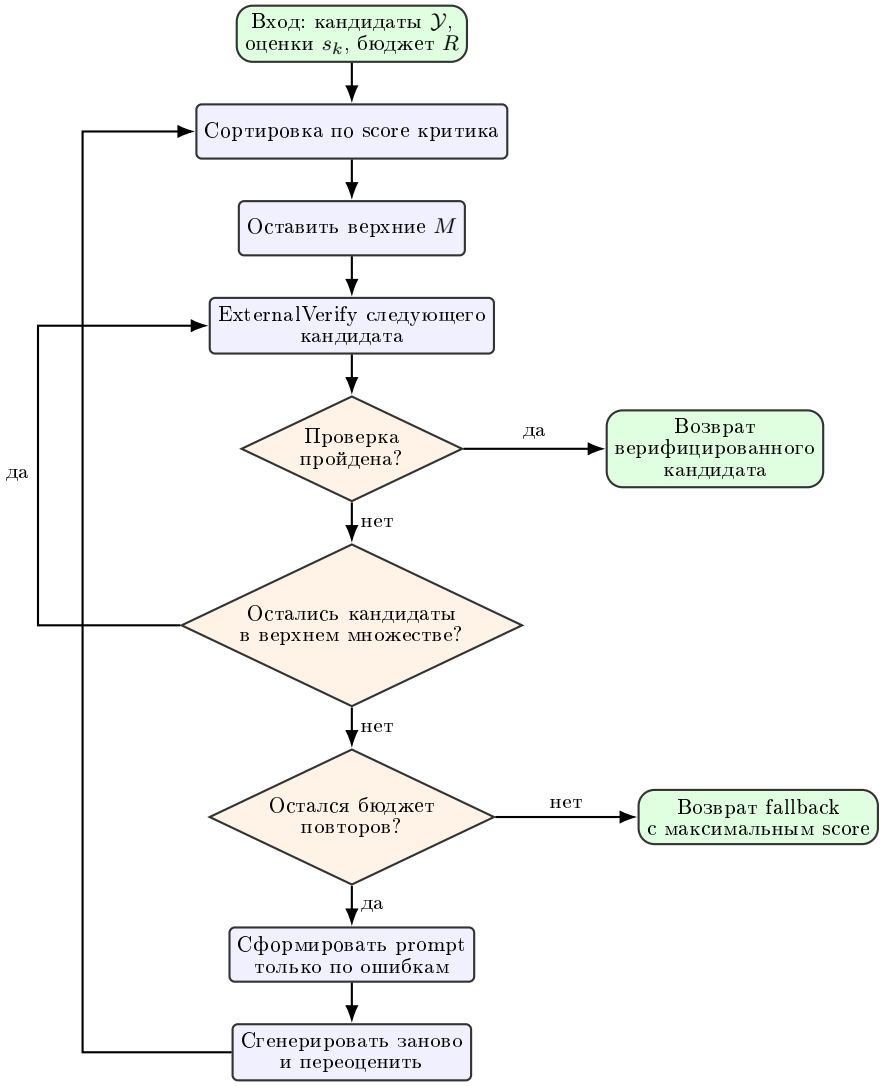

Рис. 3 Блок-схема селективного повторного запуска для алгоритма 2

4.5 Система метрик

Для неоднородного набора задач вводится агрегированное качество

$$Q = \sum_{b=1}^{B} w_b Q_b, \qquad \sum_{b=1}^{B} w_b = 1, \tag{7}$$

где $Q_b \in [0, 1]$ — нормированное качество на бенчмарке или классе задач b, а w_b — заранее объявленные веса.

Algorithm 3 Компрессия контекста для длинных задач

Require: контекст $\mathcal{C}_0$, порог длины τ_{len}
Ensure: контекст $\hat{\mathcal{C}}$

```
if |C_0| ≤ τ_len then
    return C_0
end if
Разбить C_0 на смысловые блоки
Оценить блоки по релевантности к задаче и плотности проверяемых сигналов
Удалить дубликаты, низкоинформативные логи и вторичную документацию
Сохранить сигнатуры API, failing traces, изменяемые файлы и ограничения задачи
return Ĉ
```

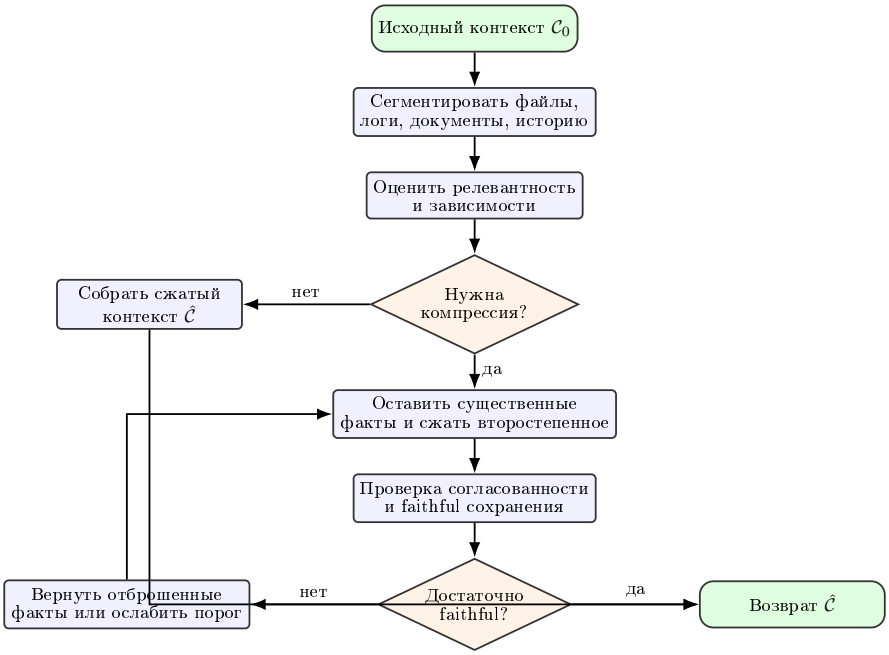

Рис. 4 Блок-схема компрессии контекста для алгоритма 3

Далее определяются основные метрики эффективности:

$$U_{\text{tok}} = \frac{Q}{T}, \tag{8}$$

$$Q_{\$} = \frac{Q}{C_{\text{inf}}}, \tag{9}$$

$$VCR = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[\texttt{tests}_i = 1 \wedge \texttt{static}_i = 1 \wedge \texttt{security}_i = 1], \tag{10}$$

Algorithm 4 Самопроверка кода и оценка поддерживаемости

Require: патч или кодовый ответ y
Ensure: метка верификации v, оценка поддерживаемости M

```
t ← RunTests(y)
s ← RunStaticAnalysis(y)
cc ← CyclomaticComplexity(y)
dup ← DuplicationRate(y)
warn ← LintWarnings(y)
doc ← DocumentationCoverage(y)
M ← w_cc(1 − c̃c) + w_dup(1 − d̃up) + w_warn(1 − w̃arn) + w_doc d̃oc
if t = pass and s = pass then
    v ← pass
else
    v ← fail
end if
return (v, M)
```

$$MAS = \frac{1}{N}\sum_{i=1}^{N} Success_i \cdot Verified_i \cdot M_i. \tag{11}$$

Оценка поддерживаемости M_i вычисляется по нормализованным прокси-показателям:

$$M_i = w_{cc}(1 - \widetilde{CC}_i) + w_{dup}(1 - \widetilde{Dup}_i) + w_{warn}(1 - \widetilde{Warn}_i) + w_{doc}\widetilde{Doc}_i, \tag{12}$$

где $\widetilde{CC}_i$ — нормализованная цикломатическая сложность [41], $\widetilde{Dup}_i$ — степень дублирования, $\widetilde{Warn}_i$ — нормализованное число предупреждений статического анализа, $\widetilde{Doc}_i$ — нормализованное покрытие кодовой документации.

Для надежности агентного контура дополнительно используются:

$$SCR = \frac{\#\{\text{некорректные кандидаты, пойманные self-check}\}}{\#\{\text{все некорректные кандидаты}\}}, \tag{13}$$

$$RFR = \frac{\#\{\text{патчи без пост-хок rollback}\}}{\#\{\text{примененные патчи}\}}, \tag{14}$$

где SCR — recall самопроверки, а RFR — rollback-free rate.

5 Планируемый экспериментальный протокол и ожидаемые показатели

5.1 Статус реализации и рамка интерпретации

На момент подготовки статьи полная версия исследуемого ассистента еще находится на стадии проектирования и поэтапной реализации. Поэтому данный раздел

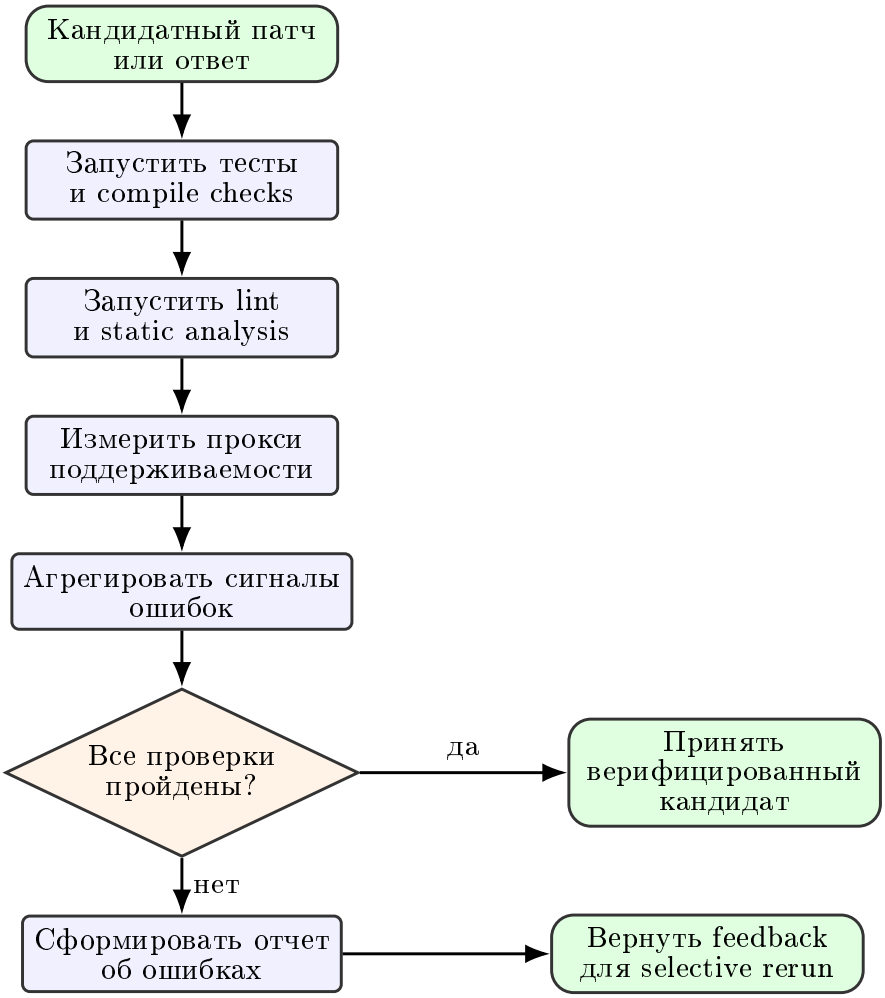

Рис. 5 Блок-схема самопроверки кода и оценки поддерживаемости для алгоритма 4

следует читать как *preregistered evaluation design*: здесь фиксируются наборы задач, baseline-конфигурации, статистические критерии и пороги интерпретации, которые будут использованы после завершения прототипа. Иными словами, ниже не сообщаются уже достигнутые экспериментальные результаты; ниже задается воспроизводимый план проверки будущих результатов.

5.2 Шкала длины запросов и классы задач

Чтобы оценка не зависела от одной произвольной длины контекста, вводится шкала:

$$short: \quad |\mathcal{C}_0| \leq 512,$$

$$
\begin{aligned}
medium: &\quad 512 < |\mathcal{C}_0| \leq 2048, \\
long: &\quad 2048 < |\mathcal{C}_0| \leq 8192, \\
very\ long: &\quad |\mathcal{C}_0| > 8192.
\end{aligned}
$$

Задачи делятся на шесть классов: *single-step*, *multi-step*, *tool-using*, *coding*, *debugging*, *document reasoning*. Для каждого класса задается ожидаемое поведение архитектуры.

Таблица 2 Классы задач и ожидаемое поведение предлагаемой архитектуры

Класс задачи	Основной режим	Ожидаемое поведение
single-step	внутренний	Один кандидат, облегченный critic, без внешних инструментов; интроспекция минимальна, чтобы не проиграть одноагентному baseline по токенам.
multi-step	внутренний	Planner задает план, worker генерирует несколько гипотез, critic выявляет логические пропуски.
tool-using	гибридный	Policy controller определяет допустимые инструменты; self-check обязателен перед действием записи или исполнения.
coding	гибридный	Worker формирует патч, self-check запускает тесты, статический анализ и maintainability scoring.
debugging	гибридный	Сначала компрессия failing traces и релевантных файлов, затем несколько кандидатов исправления и селективный rerun.
document reasoning	внутренний	Memory/compression устраняет шум длинной документации; critic проверяет полноту и противоречия цитируемых фрагментов.

5.3 Наборы данных и синтетические, но воспроизводимые расширения

Базовый протокол включает три публичных набора:

1. **HumanEval** для коротких задач генерации кода [18];
2. **MBPP** для базовых программных задач естественного языка [19];
3. **SWE-bench** для задач исправления дефектов в реальных репозиториях [16].

Эти наборы дополняются двумя *synthetic but reproducible* семействами, потому что публичные benchmarks недостаточно покрывают длинный документный контекст и контролируемые tool-use сценарии:

1. **Synthetic Tool-Use Suite**: задачи генерируются из конечной грамматики действий над файлами, командной строкой и структурированными артефактами; для каждой задачи создается детерминированный oracle правильной последовательности.
2. **Synthetic Document Reasoning Suite**: из публично разрешенных технических текстов формируются задачи на сравнение версий, выявление противоречий, составление кратких решений и проверку соблюдения ограничений; генератор фиксируется seed-значениями и публикуется вместе с шаблонами.

5.4 Базовые конфигурации и статистика

Сравнение проводится в четырех режимах: **B0 Single-Large**, **B1 Single-Large+Self-Refine**, **B2 Classical-MAS** и **B3 Hybrid-INoT**. Дополнительные абляции включают *no-critic*, *no-compression*, *no-self-check*, *large-model-only* и *always-external*. Для стохастических режимов каждая конфигурация запускается как минимум с пятью случайными seed-значениями на HumanEval, MBPP и синтетических наборах; для дорогих задач исправления дефектов допускается не менее трех запусков при условии полного раскрытия идентификаторов задач и seed-значений.

5.5 Ожидаемые эффекты и критерии будущей проверки

Чтобы не смешивать гипотезы, инженерные ожидания и уже наблюденные данные, в таблице 3 фиксируются *не результаты*, а предрегистрационные пороги и ожидаемые направления эффекта. Если последующая реализация ассистента не подтвердит эти ориентиры, гипотезы должны считаться неподтвержденными или частично подтвержденными.

Отчет включает средние значения с 95% доверительными интервалами, bootstrap-интервалы по задачам с не менее чем 10 000 ресэмплирований, парный критерий Мак-Немара для бинарных исходов успеха, критерий Вилкоксона для попарного сравнения токенов, стоимости и задержки, а также поправку Holm–Bonferroni при множественных сравнениях. Закрытые API-модели без неизменяемого snapshot ID выносятся в отдельное observational appendix и не смешиваются с воспроизводимым треком.

6 Экономический анализ и TCO

6.1 Формализация совокупной стоимости владения

Для месячного горизонта расчета совокупная стоимость владения определяется как

$$TCO_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}, \tag{15}$$

где

$$C_{\text{hw}} = \frac{CapEx_{\text{cpu}} + CapEx_{\text{gpu}} + CapEx_{\text{ram}}}{L_{\text{amort}}}, \tag{16}$$

Таблица 3 Ожидаемые направления эффекта и минимальные practically relevant thresholds для будущей проверки

Метрика	Режим / класс задач	Предрегистрационное ожидание или порог
Q и success rate	*coding*, *debugging*, *document reasoning* на *long/very long*	Hybrid-INoT должен быть как минимум не хуже Classical-MAS в пределах non-inferiority margin 2 процентных пунктов по основной метрике успеха.
U_{tok}	*long/very long* задачи с общим контекстом	Практически значимым улучшением считается рост не менее чем на 15% относительно Classical-MAS.
$Q_{\$}$	те же режимы	Практически значимым улучшением считается рост не менее чем на 10% относительно Classical-MAS.
Latency	*short* vs. *long*	Для *short* задач ухудшение более чем на 10% считается признаком неудачной маршрутизации; для *long* задач ожидается нейтральный или умеренно положительный эффект за счет уменьшения числа дорогих rerun.
VCR	*coding*, *debugging*	По сравнению с B0 Single-Large ожидается прирост не менее чем на 5 процентных пунктов за счет внешней самопроверки.
MAS	*coding*	Успех не должен достигаться ценой деградации maintainability: ухудшение более чем на 5% относительно Classical-MAS считается неприемлемым.

$$C_{\text{energy}} = P_{\text{avg}} \cdot h_{\text{month}} \cdot c_{\text{kWh}}, \tag{17}$$

$$C_{\text{ops}} = C_{\text{monitoring}} + C_{\text{maintenance}} + C_{\text{human-review}}. \tag{18}$$

6.2 Синтетический, но воспроизводимый сценарий развертывания

Таблица 4 представляет *синтетический*, а не эмпирически измеренный сценарий. Ее цель состоит не в том, чтобы заявить достигнутый выигрыш как факт, а в том, чтобы показать, как архитектуры следует сравнивать при явно объявленных предпосылках.

При этих явно объявленных предпосылках развернутый расчет дает снижение синтетического месячного TCO на 23.4%. Это не эмпирическое измерение на уже развернутой производственной системе, а воспроизводимый шаблон расчета, который позднее должен быть заменен наблюдаемыми данными о реальном развертывании.

6.3 Когда подход выгоден и когда нет

Предлагаемая архитектура вероятнее всего экономически выгодна, если задачи содержат длинный общий контекст, который в Classical-MAS пришлось бы многократно пересылать независимым ролям, если значимая доля ошибок может быть поймана дешевой самопроверкой до полного перезапуска крупной модели, если параллельная работа ролей не является критичным источником ускорения, а малая модель или внешняя проверка достаточно надежны, чтобы уменьшать ρ в

Таблица 4 Синтетический, но воспроизводимый пример месячного TCO для стендового развертывания

Статья затрат / допущение	Classical-MAS	Hybrid-INoT
Число задач в месяц	10,000	10,000
Ожидаемые input токены крупной модели / задача	22,000	9,000
Ожидаемые output токены крупной модели / задача	3,200	1,800
Ожидаемые токены малых помощников / задача	0	1,800
Синтетическая стоимость инференса, USD / месяц	664	497
Амортизация CPU/RAM-сервера, USD / месяц	220	220
Амортизация GPU-ускорителей, USD / месяц	360	180
Storage и резервные артефакты, USD / месяц	20	20
Энергопотребление, USD / месяц	70	35
Сопровождение и human review, USD / месяц	300	300
Итого синтетический TCO, USD / месяц	1,634	1,252

формуле (6). Подход может быть невыгоден при обратных условиях: очень короткие *single-step* задачи, высокий выигрыш от истинного параллелизма, слабая способность модели следовать ролевому формату, а также сценарии, где безопасность требует жесткой изоляции каждого инструмента и каждой привилегии в отдельном процессе.

7 Обсуждение

7.1 Почему интроспекция мыслей особенно важна для AI-агентов

Для обычного одношагового чат-ответа промежуточная интроспекция может быть лишь способом улучшить финальный текст. Для AI agents значение выше: каждое промежуточное решение влияет на выбор инструмента, объем следующего контекста, число последующих итераций и, в конечном счете, на цену всей траектории. Следовательно, интроспекция здесь выполняет одновременно функцию управления качеством и функцию управления стоимостью.

7.2 Надежность кода, самопроверка и поддерживаемость

Ключевой инженерный принцип статьи состоит в отказе от допущения, что правдоподобный текстовый ответ автоматически означает надежное изменение кода. Верификация должна оставаться внешней и наблюдаемой: тесты, компилятор, линтеры, статический анализ и проверки безопасности. Этот тезис согласуется с работами по compiler feedback и статическому анализу в кодогенерации [23, 24]. Поддерживаемость тоже рассматривается как первичная инженерная цель. Решение, которое проходит тесты лишь ценой резкого роста цикломатической сложности, дублирования или числа предупреждений анализа, не должно считаться полным успехом.

7.3 Ограничения метода и угрозы валидности

Предлагаемый подход имеет несколько существенных ограничений. Внутренний ролевой текст не гарантирует достоверного объяснения скрытого вычислительного процесса модели [9]. На задачах с подлинной независимостью ролей внешняя мультиагентная архитектура может быть предпочтительнее. Компрессия контекста сама по себе может удалить редкий, но критичный факт, если селектор релевантности спроектирован неудачно. Метрики поддерживаемости зависят от выбранного набора статических прокси и не заменяют полноценное архитектурное ревью. Наконец, закрытые API-модели подвержены дрейфу версии; без фиксированных snapshot ID они не подходят для строгого воспроизводимого трека.

8 Заключение

Настоящая версия статьи задает не финальный отчет об уже завершенной системе, а методологически строгую исследовательскую рамку для ее последующей проверки. В работе очищен библиографический и фактический контур, уточнен реальный исследовательский зазор, формализован Hybrid-INoT как гибридная агентная архитектура и введен набор метрик, позволяющий проверять не только качество ответа, но и токенную экономику, стоимость инференса, надежность кода и поддерживаемость.

Практический смысл этой рамки состоит в том, что после завершения прототипа ассистента одноагентные baseline-режимы, классическую внешнюю мультиагентную оркестрацию и внутреннюю ролевую интроспекцию можно будет сравнить на одних и тех же задачах и по заранее объявленным критериям интерпретации. Именно такая сравнительная и проверяемая постановка, а не преждевременное заявление о превосходстве подхода, и составляет основной научный результат текущей стадии проекта.

References

[1] H. Sun and S. Zeng, “Introspection of Thought Helps AI Agents,” arXiv preprint arXiv:2507.08664, 2025.

[2] A. J. Ko, R. DeLine, and G. Venolia, “Information Needs in Collocated Software Development Teams,” in *Proceedings of the 29th International Conference on Software Engineering*, 2007, pp. 344–353.

[3] C. Parnin and S. Rugaber, “Resumption Strategies for Interrupted Programming Tasks,” *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.

[4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, “Collaboration, Information Seeking and Communication: An Observational Study of Software Developers’ Work Practices,” *Journal of Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.

[5] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.

[6] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv preprint arXiv:2210.03629, 2022.

[7] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," arXiv preprint arXiv:2303.17651, 2023.

[8] N. Shinn *et al.*, "Reflexion: Language Agents with Verbal Reinforcement Learning," arXiv preprint arXiv:2303.11366, 2023.

[9] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," arXiv preprint arXiv:2305.04388, 2023.

[10] G. Weiss, *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*. Cambridge, MA, USA: MIT Press, 1999.

[11] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: Wiley, 2009.

[12] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," arXiv preprint arXiv:2308.00352, 2023.

[13] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," arXiv preprint arXiv:2307.07924, 2023.

[14] T. Guo *et al.*, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," arXiv preprint arXiv:2402.01680, 2024.

[15] C. Qian *et al.*, "Scaling Large Language Model-based Multi-Agent Collaboration," arXiv preprint arXiv:2406.07155, 2024.

[16] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," arXiv preprint arXiv:2310.06770, 2023.

[17] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," arXiv preprint arXiv:2405.15793, 2024.

[18] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[19] J. Austin *et al.*, "Program Synthesis with Large Language Models," arXiv preprint arXiv:2108.07732, 2021.

[20] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," arXiv preprint arXiv:2310.06839, 2023.

[21] Z. Pan *et al.*, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," arXiv preprint arXiv:2403.12968, 2024.

[22] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing Code Context for LLM-based Issue Resolution," arXiv preprint arXiv:2603.28119, 2026.

[23] Z. Bi *et al.*, "Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback," arXiv preprint arXiv:2403.16792, 2024.

[24] C.-Y. Su and C. McMillan, "Do Code LLMs Do Static Analysis?," arXiv preprint arXiv:2505.12118, 2025.

[25] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, "Enhancing LLM-Based Code Generation with Complexity Metrics: A Feedback-Driven Approach," arXiv preprint arXiv:2505.23953, 2025.

[26] DeepSeek-AI *et al.*, "DeepSeek-V3 Technical Report," arXiv preprint arXiv:2412.19437, 2024.

[27] DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv preprint arXiv:2501.12948, 2025.

[28] Z.ai, "GLM-4.5: Agentic, Reasoning, and Coding Foundation Models," arXiv preprint arXiv:2508.06471, 2025.

[29] Z.ai, "GLM-4.5," 2025. [Online]. Available: https://docs.z.ai/guides/llm/glm-4.5

[30] Google, "Gemini 3.1 Pro," Feb. 19, 2026. [Online]. Available: https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro

[31] Google, "Gemini API models," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs/models/gemini

[32] Anthropic, "Claude Sonnet 4.5," Sep. 29, 2025. [Online]. Available: https://www.anthropic.com/news/claude-sonnet-4-5

[33] Anthropic, "Extended thinking," 2026. [Online]. Available: https://docs.claude.com/en/docs/build-with-claude/extended-thinking

[34] MiniMax, "Text generation models and guides," 2026. [Online]. Available: https://platform.minimax.io/docs/api-reference/text-intro

[35] Xiaomi, "MiMo," 2026. [Online]. Available: https://mimo.mi.com/

[36] OpenAI, "Introducing ChatGPT agent: bridging research and action," Jul. 17, 2025. [Online]. Available: https://openai.com/index/introducing-chatgpt-agent/

[37] OpenAI, "ChatGPT agent System Card," Jul. 17, 2025. [Online]. Available: https://openai.com/index/chatgpt-agent-system-card/

[38] OpenAI, "Agents SDK," 2026. [Online]. Available: https://platform.openai.com/docs/guides/agents-sdk/

[39] OpenAI, "A Practical Guide to Building Agents," 2026. [Online]. Available: https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf

[40] ISO/IEC 25010:2023, "Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model," International Organization for Standardization, 2023.

[41] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.